

Checklist

Platform Engineering Groundwork

1. Establish the Platform Foundation

- Treat the engineering platform as a product focused on Developer Experience (DevEx).
- Identify the developers, SREs, security/compliance teams, product managers, and other stakeholders who will use or influence the platform.
- Establish short feedback loops with platform users.
- Define metrics for adoption, reliability, delivery performance, and developer satisfaction.
- Establish supported **golden paths** and sensible defaults for developers.
- Embed security, governance, testing, and compliance into the platform from the beginning.

2. Prepare the Core Tooling

- Install/configure **Pulumi with Python and uv** for Infrastructure as Code.
- Configure **Bitwarden** for secrets management.
- Configure **GitHub** for source control and repository management.
- Configure **CircleCI** and a local runner for CI/CD.
- Have **Kind and Helm** available for later Kubernetes work.
- Prepare the Python quality toolchain, including pytest, ruff, mypy, and bandit.

3. Define Repository Architecture

- Decide deliberately between a **monorepo** and **polyrepo** approach.
- Identify the platform's domain boundaries.
- Separate repositories or deployment pipelines by platform domain where appropriate.
- Establish consistent repository naming conventions.

- Establish branch naming conventions.
- Define repository access roles.
- Define repository policies and governance controls.
- Ensure security and compliance guardrails span all platform domains.

4. Create the Platform Administration Project

- Create the `platform-team-administration` directory.
- Run `pulumi new python`.
- Select `uv` for package management.
- Create the standard project folders shown in the chapter, including configuration, modules, scripts, secrets setup, and tests.
- Ensure the project structure can be reproduced by another platform engineer.

5. Configure Secrets Management

- Create `.env_example`.
- Add placeholders for `BW_CLIENTID`, `BW_CLIENTSECRET`, and `BW_PASSWORD`.
- Copy `.env_example` to `.env`.
- Insert the actual Bitwarden credentials into `.env`.
- Add `.env` to `.gitignore`.
- Create `github_secrets.json_example`.
- Create the local `github_secrets.json` containing the actual values.
- Add the real secrets JSON file to `.gitignore`.
- Store the GitHub Personal Access Token in Bitwarden.
- Store the GitHub organization/owner information required by Pulumi.
- Create `inject_secrets.sh`.
- Make the script executable.
- Authenticate with the Bitwarden CLI.
- Unlock the vault and obtain a session.
- Make the script update an existing Bitwarden item or create it when absent.
- Sync Bitwarden after injecting secrets.
- Lock the vault after the operation completes.
- Verify that the secrets appear correctly in Bitwarden.

Exercise — Pulumi Secret

- Create a Bitwarden-compatible JSON template for the Pulumi API key.
- Name the secret `Pulumi Secrets`.

- Create the field `pulumi-api-key`.
- Store the Pulumi API key in that field.

6. Automate Repository Creation

- Create `config/platform_team_values.yaml`.
- Make the configuration file the source of truth for managed repositories.
- Define `platform-team-admin`.
- Define `platform-core`.
- Define `platform-demo-apps`.
- Configure the Pulumi GitHub provider.
- Retrieve the required GitHub credentials from the secrets configuration.
- Load repository definitions from the YAML configuration.
- Create repositories programmatically with Pulumi.
- Support repository visibility through configuration.
- Run `pulumi preview`.
- Review the proposed changes.
- Run `pulumi up`.
- Verify that the repositories were created in GitHub.

Exercise — Repository Delete Protection

- Add delete protection to the Pulumi-managed repositories.
- Confirm that accidental `pulumi destroy` operations cannot delete protected repositories.
- Add a new repository named `platform-extensions`.
- Deploy `platform-extensions` to GitHub.

7. Automate Platform-Team Onboarding

- Create a second GitHub user for testing onboarding.
- Add `github_organization_members` to `platform_team_values.yaml`.
- Define each member's name.
- Define their GitHub username.
- Define their GitHub role.
- Define their email where required.
- Update the Pulumi code to process organization members.
- Run `pulumi up`.
- Verify that the new team member receives the organization invitation.

- Prefer administrative access through a platform service account or group rather than individual engineers.

Exercise — Offboarding

- Remove the test team member from the IaC configuration.
- Apply the configuration.
- Verify that the user can no longer access the organization's repositories.
- Change a test user's role from `admin` to `member`.
- Verify how the user's permissions change.

8. Establish Commit Conventions

- Define the required commit-message structure.
- Define allowed commit types such as `feat`, `fix`, `docs`, `test`, `refactor`, and others.
- Define how commit scopes will be used.
- Establish tagging/versioning conventions.
- Create a `.git-hooks/commit-msg` hook.
- Validate commit messages against the Conventional Commits-style format.
- Create `scripts/install-githooks.sh`.
- Have the installation script copy the hook into `.git/hooks/commit-msg`.
- Add pre-commit checks for linting, formatting, or other fast validations where appropriate.
- Plan to automate changelog generation and version bumps.

9. Enforce Repository Policies

- Agree on repository standards with the team.
- Establish branch-protection requirements.
- Establish merge/pull-request requirements.
- Configure GitHub policy enforcement through IaC.
- Require cryptographically signed commits.
- Apply the protection policy to the repository.
- Ensure administrators are also subject to the appropriate protection rules.
- Create/configure an SSH or GPG signing certificate for GitHub.
- Verify that unsigned commits are rejected.
- Verify that properly signed commits succeed.

Exercise 1.1 — Expand Repository Policy

- Modify the signed-commit policy so it applies to **all branches**, not only `main`.
- Verify that commits on every branch can be traced to an authorized developer.

10. Establish the Branching and Release Strategy

- Adopt a deliberate branching strategy.
- Use trunk-based development as described in the chapter.
- Have changes integrate through `main`.
- Trigger validation/deployment processes from commits to `main`.
- Establish a release-tagging convention.
- Separate **validation on push** from **release on tag**.

11. Configure CircleCI

- Create `.circleci/config.yml`.
- Configure CircleCI version `2.1`.
- Add the Pulumi orb.
- Add the Python orb.
- Configure filters for pushes to `main`.
- Configure separate filters for tags.
- Configure the local CircleCI runner.
- Create the `PLATFORM_ADMIN` CircleCI context.
- Make required Bitwarden/environment values available through the context.

12. Build the Release Workflows

- Create a **preview** workflow for pushes.
- Run `pulumi-preview` on qualifying pushes.
- Create an **update/release** workflow for tags.
- Run `pulumi-preview` before a tagged deployment.
- Add an approval gate before infrastructure changes are applied.
- Run `pulumi-update` only after approval.
- Include automated testing before deployment.
- Include configuration validation before deployment.
- Consider automated rollback triggers based on health checks.
- Establish a process for regularly testing rollback mechanisms.

13. Exercise 1.2 — Create the First Release

- Ensure all changes are committed.
- Ensure the local branch is up to date.
- Create the annotated tag:

```
git tag -a v0.1.0 -m "First stable release of platform setup"
```
- Push the tag:

```
git push origin v0.1.0
```
- Verify that the tag appears in GitHub under Releases.
- Verify that the tag-triggered CircleCI workflow runs as expected.

Chapter 1 Completion Check

- Repository structure is defined and repeatable.
- Secrets are kept out of source control.
- Repository provisioning is automated.
- Developer onboarding/offboarding is configuration-driven.
- Commit conventions are defined and validated.
- Signed-commit policies are enforced.
- Trunk-based development and release tagging are established.
- Pushes trigger validation.
- Tags trigger the controlled release workflow.
- The first stable release tag has been created and verified.

Chapter 1 complete: the platform now has the repository structure, commit conventions, policy automation, and release-tagging foundation needed for the next stage of the engineering platform.